Logo entreprise

ÉCOLE SUPÉRIEURE PRIVÉE D'INGÉNIERIE ET DE TECHNOLOGIES
ESPRIT

Rapport de stage de fin d'études

en vue de l'obtention du
Diplôme National d'Ingénieur en Informatique

Spécialité : Génie Logiciel – option **[À compléter : à confirmer : option / classe]**

Conception et réalisation de MediBook : plateforme SaaS de prise de rendez-vous et de suivi des patients pour les cliniques

---

Réalisé par : **Prénom NOM**

Encadrant entreprise : M. Prénom NOM, directeur technique de Nova Solutions
Encadrant académique : Mme Prénom NOM

Organisme d'accueil : **Nova Solutions**

Stage du 2 février 2027 au 31 juillet 2027

Année universitaire 2026 – 2027

Dédicace

À ma famille, pour son soutien constant tout au long de mes études.

À mes amis, et à toutes les personnes qui m'ont accompagné pendant ce projet.

Prénom NOM

Remerciements

Au terme de ce stage de fin d'études, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à la réussite de ce travail.

Je remercie tout particulièrement M. Prénom NOM, directeur technique de Nova Solutions, mon encadrant au sein de Nova Solutions, pour sa confiance, sa disponibilité et la qualité de ses conseils tout au long du projet.

Je remercie également Mme Prénom NOM, mon encadrante académique à ESPRIT, pour son suivi rigoureux, ses remarques constructives et son accompagnement pendant les différentes restitutions.

Mes remerciements s'adressent aussi à toute l'équipe de Nova Solutions pour son accueil, ses revues de code et les nombreux échanges techniques qui ont enrichi ce travail.

Enfin, je remercie les membres du jury d'avoir accepté d'évaluer ce travail.

Table des matières

Liste des abréviations **vii**

Introduction générale **1**

1 Contexte général du projet **2**

1.1 Présentation de l'organisme d'accueil 2

1.1.1 Activités et services 2

1.2 Cadre du projet 2

1.3 Étude de l'existant 3

1.3.1 Critique de l'existant 3

1.4 Problématique 3

1.5 Solution proposée 3

1.6 Méthodologie de travail 4

1.7 Planning prévisionnel 4

2 État de l'art et choix technologiques **5**

2.1 Concepts clés 5

2.1.1 Stratégies de multi-location 5

2.2 Étude des solutions existantes 5

2.3 Choix technologiques 6

3 Analyse et spécification des besoins **7**

3.1 Identification des acteurs 7

3.2 Besoins fonctionnels 7

3.3 Besoins non fonctionnels 8

3.4 Diagramme de cas d'utilisation global 8

3.5 Backlog du produit 9

4 Architecture et conception **11**

4.1 Architecture globale 11

4.1.1 Isolation des cliniques 11

4.2 Conception détaillée 12

4.2.1 Diagramme de classes 12

4.2.2 Diagramme de séquence : prise de rendez-vous 14
4.2.3 Diagramme d'activité : prise de rendez-vous 15
4.2.4 Diagramme d'états : rendez-vous 15

5 Réalisation **17**
5.1 Environnement de travail 17
5.2 Release 1 : socle et agendas 17
5.2.1 Backlog de la release 17
5.2.2 Interfaces réalisées 18
5.3 Release 2 : réservation et rappels 18
5.3.1 Backlog de la release 18
5.3.2 Interfaces réalisées 18
5.4 Release 3 : pilotage et mise en production 19
5.4.1 Backlog de la release 19
5.4.2 Interfaces réalisées 19

6 Tests, qualité et déploiement **20**
6.1 Stratégie de test 20
6.2 Résultats 20
6.3 Intégration et livraison continues 21
6.4 Déploiement 21

Conclusion générale et perspectives **22**

Bibliographie et netographie **23**

A Planning détaillé et rendez-vous pédagogiques **24**

B Extraits de configuration **25**
B.1 Filtre d'isolation des cliniques 25
B.2 Déploiement Kubernetes 25

C Design Thinking : brainstorming et prototype **26**
C.1 Séance de brainstorming 26
C.2 Prototype basse fidélité 28

Liste des figures

1.1 Planning prévisionnel du stage (en semaines) 4

3.1 Diagramme de cas d'utilisation global 9

4.1 Architecture globale de MediBook 11
4.2 Diagramme de classes d'analyse 13
4.3 Diagramme de séquence : prise de rendez-vous 14
4.4 Diagramme d'activité : prise de rendez-vous 15
4.5 Diagramme d'états d'un rendez-vous 15

5.1 Agenda hebdomadaire d'un praticien 18
5.2 Réservation d'un créneau par le patient 18
5.3 Tableau de bord d'une clinique 19

6.1 Chaîne d'intégration et de livraison continues 21
6.2 Diagramme de déploiement 21

C.1 Tableau de brainstorming de MediBook 27
C.2 Prototype basse fidélité de MediBook 28

Liste des tableaux

1.1 Pôles d'activité de Nova Solutions 2
1.2 Rôles Scrum dans le projet 4

2.1 Stratégies d'isolation des données 5
2.2 Comparaison des solutions existantes 6
2.3 Choix technologiques 6

3.1 Acteurs de la plateforme 7
3.2 Besoins fonctionnels par module 7
3.3 Besoins non fonctionnels 8
3.4 Extrait du backlog du produit 9

5.1 Environnement de développement 17

6.1 Niveaux de test 20
6.2 Résultats des tests (valeurs d'exemple) 20

A.1 Planning détaillé du stage (26 semaines) 24

Liste des abréviations

API	*Application Programming Interface*
CI/CD	Intégration et livraison continues
DPI	Dossier patient informatisé
E2E	Tests de bout en bout (*end-to-end*)
JWT	*JSON Web Token*
OIDC	*OpenID Connect*
ORM	*Object-Relational Mapping*
RBAC	*Role-Based Access Control* – contrôle d'accès par rôles
RGPD	Règlement général sur la protection des données
SaaS	*Software as a Service*
SMS	*Short Message Service*
SPA	*Single Page Application*
UI	Interface utilisateur
UML	*Unified Modeling Language*

Introduction générale

La transformation numérique du secteur de la santé s'accélère. Les patients attendent aujourd'hui de pouvoir réserver une consultation aussi simplement qu'un billet de train, à toute heure et sans passer par un appel téléphonique. Du côté des cliniques, la gestion des agendas reste pourtant souvent manuelle : registres papier, tableurs partagés et appels répétés au secrétariat.

Cette situation a un coût. Les secrétariats passent une grande partie de leur journée au téléphone, les praticiens subissent des créneaux vides lorsque les patients oublient leur rendez-vous, et la direction ne dispose d'aucun indicateur fiable sur l'occupation des cabinets.

C'est dans ce contexte que s'inscrit notre projet de fin d'études, réalisé au sein de Nova Solutions du 2 février 2027 au 31 juillet 2027. Il a pour objectif de concevoir et de réaliser MediBook, une plateforme SaaS qui permet à plusieurs cliniques de publier leurs disponibilités, aux patients de réserver en ligne, et à chaque établissement de suivre son activité grâce à un tableau de bord.

Le présent rapport s'articule autour de six chapitres :

- le premier chapitre présente l'organisme d'accueil, le contexte du projet, l'étude de l'existant et la méthodologie adoptée ;
- le deuxième chapitre expose l'état de l'art, les solutions du marché et les choix technologiques ;
- le troisième chapitre détaille l'analyse et la spécification des besoins ;
- le quatrième chapitre décrit l'architecture et la conception de la solution ;
- le cinquième chapitre présente la réalisation, sprint par sprint ;
- le sixième chapitre traite des tests, de la qualité et du déploiement.

Nous clôturons ce rapport par une conclusion générale qui résume le travail réalisé et ouvre des perspectives.

Chapitre 1

Contexte général du projet

Introduction

Ce chapitre présente l'organisme d'accueil et le cadre du projet. Il décrit ensuite l'existant et ses limites, formule la problématique, puis présente la solution proposée, la méthodologie de travail et le planning prévisionnel.

1.1 Présentation de l'organisme d'accueil

Nova Solutions est une entreprise de services numériques fondée en 2018 et basée à Tunis. Elle compte une quarantaine de collaborateurs et développe des applications web et mobiles sur mesure pour des clients du secteur de la santé et de la distribution. **[À compléter : Remplacer par la présentation réelle de votre entreprise : historique, effectif, clients, organigramme.]**

1.1.1 Activités et services

Le tableau 1.1 résume les principaux pôles d'activité de l'entreprise.

Tab. 1.1. Pôles d'activité de Nova Solutions

Pôle	Description
Développement web	Applications métier, portails clients, plateformes SaaS.
Développement mobile	Applications Android et iOS natives ou multiplateformes.
Conseil et DevOps	Audit d'architecture, conteneurisation, mise en place de chaînes CI/CD.
Maintenance	Tierce maintenance applicative et support de niveau 2.

1.2 Cadre du projet

Ce projet est réalisé dans le cadre du stage de fin d'études en vue de l'obtention du diplôme national d'ingénieur en informatique à ESPRIT. Il répond à un besoin exprimé par plusieurs cliniques partenaires de Nova Solutions, qui souhaitent proposer la prise de rendez-vous en ligne à leurs patients.

1.3 Étude de l'existant

Aujourd'hui, les cliniques partenaires gèrent leurs rendez-vous de trois façons : un registre papier au secrétariat, un tableur partagé entre les secrétaires, ou un logiciel de cabinet installé sur un seul poste. Le patient appelle la clinique, la secrétaire cherche un créneau libre et le note à la main. Une séance de brainstorming avec l'équipe de Nova Solutions a permis de recenser les problèmes, les utilisateurs et les premières idées de solution (annexe C).

1.3.1 Critique de l'existant

Cette organisation présente plusieurs limites :

- **Disponibilité** : la réservation n'est possible qu'aux heures d'ouverture du secrétariat ;
- **Charge du secrétariat** : une part importante de la journée est consacrée aux appels ;
- **Rendez-vous non honorés** : sans rappel, une partie des patients oublie leur rendez-vous ; **[À compléter : citer un chiffre mesuré dans l'entreprise]**
- **Absence d'indicateurs** : la direction ne connaît ni le taux d'occupation ni le délai moyen d'attente ;
- **Données dispersées** : chaque clinique possède ses propres fichiers, sans sauvegarde centralisée.

1.4 Problématique

Comment offrir à plusieurs cliniques, sur une même plateforme, un service de prise de rendez-vous en ligne fiable et sécurisé, qui réduise la charge du secrétariat et les rendez-vous non honorés, tout en garantissant l'isolation et la confidentialité des données de chaque établissement ?

1.5 Solution proposée

Nous proposons MediBook, une plateforme SaaS multi-cliniques composée de trois espaces :

- un **espace patient** pour rechercher un praticien, réserver, déplacer ou annuler un rendez-vous ;
- un **espace clinique** pour gérer les praticiens, leurs agendas, les rendez-vous et les rappels ;
- une **console d'administration** pour créer les cliniques, gérer les abonnements et superviser la plateforme.

1.6 Méthodologie de travail

Définition : Scrum

Scrum est un cadre de travail agile dans lequel le produit est construit par itérations courtes et de durée fixe, appelées sprints, à partir d'un backlog priorisé par le *Product Owner* [3].

Nous avons adopté Scrum avec des sprints de deux semaines. Le tableau 1.2 présente la répartition des rôles.

Tab. 1.2. Rôles Scrum dans le projet

Rôle	Personne
Product Owner	M. Prénom NOM, directeur technique de Nova Solutions
Scrum Master	Chef de projet de Nova Solutions
Équipe de développement	Prénom NOM (stagiaire PFE)

1.7 Planning prévisionnel

La figure 1.1 présente le planning prévisionnel du stage sur 26 semaines. Le planning détaillé figure en annexe A.

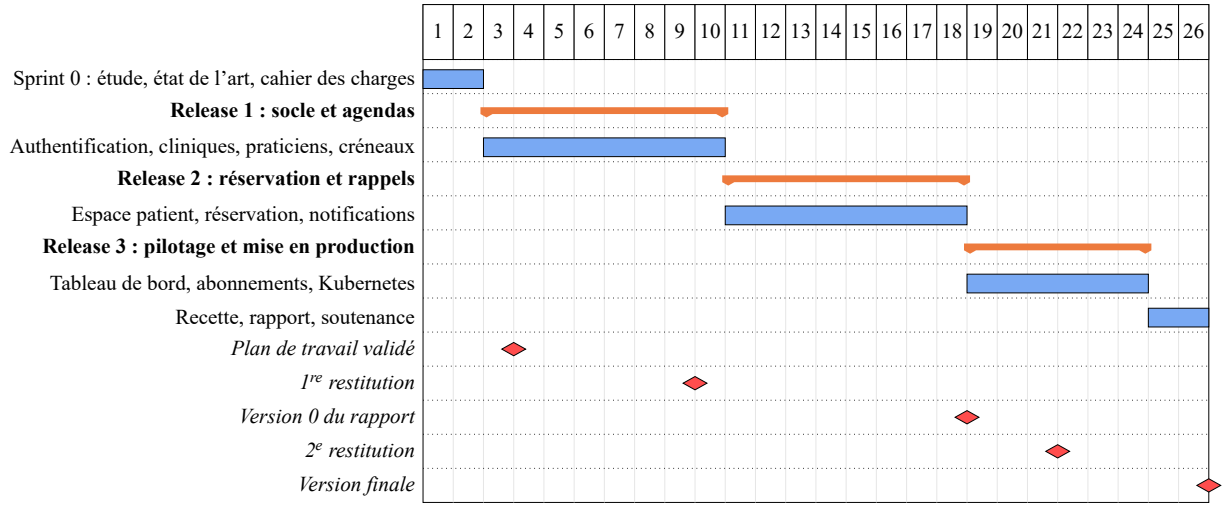


Fig. 1.1. Planning prévisionnel du stage (en semaines)

Conclusion

Ce chapitre a présenté l'organisme d'accueil, le cadre du projet et l'existant. Il a montré que la gestion manuelle des rendez-vous limite la disponibilité du service et ne fournit aucun indicateur. Avant de spécifier la solution, le chapitre suivant étudie l'état de l'art et les solutions du marché.

Chapitre 2

État de l'art et choix technologiques

Introduction

Ce chapitre définit les concepts clés du projet, compare les solutions de prise de rendez-vous existantes, puis justifie les choix technologiques retenus.

2.1 Concepts clés

Définition : Logiciel en tant que service (SaaS)

Modèle de distribution dans lequel l'application est hébergée par l'éditeur et utilisée par ses clients à travers le web, sur abonnement, sans installation locale.

Définition : Multi-location

Architecture dans laquelle une seule instance de l'application sert plusieurs clients (les « locataires », ici les cliniques), en isolant strictement leurs données.

2.1.1 Stratégies de multi-location

Le tableau 2.1 compare les trois stratégies d'isolation des données les plus répandues.

Tab. 2.1. Stratégies d'isolation des données

Stratégie	Avantages	Inconvénients	Coût
Base par locataire	Isolation maximale	Exploitation lourde	Élevé
Schéma par locataire	Bon compromis	Migrations multipliées	Moyen
Colonne `clinic_id`	Simple, économique	Isolation à garantir dans le code	Faible

2.2 Étude des solutions existantes

Nous avons étudié trois solutions du marché selon des critères définis avec l'encadrant entreprise (tableau 2.2). **[À compléter : Remplacer « Solution A/B/C » par les vrais produits étudiés et citer leurs sites dans la bibliographie.]**

Tab. 2.2. Comparaison des solutions existantes

Critère	Solution A	Solution B	Solution C	MediBook
Réservation en ligne 24 h/24	✓	✓	✓	✓
Multi-cliniques	✓	–	✓	✓
Rappels SMS	✓	✓	–	✓
Interface en arabe	–	–	–	✓
Hébergement des données en Tunisie	–	–	–	✓
Tarif adapté au marché local	–	✓	–	✓

2.3 Choix technologiques

Le tableau 2.3 résume les technologies retenues et la raison de chaque choix.

Tab. 2.3. Choix technologiques

Couche	Technologie	Justification
Back-end	Spring Boot 3 (Java 21)	Maîtrise de l'équipe, écosystème mature, sécurité intégrée [6].
Front-end	React 18 + TypeScript	Composants réutilisables, typage statique [2].
Base de données	PostgreSQL 16	Robustesse, contraintes d'intégrité, index partiels [5].
Authentification	Keycloak	OIDC standard, gestion des rôles, fédération d'identités.
Messagerie	RabbitMQ	Envoi asynchrone des rappels, reprise en cas d'échec.
Conteneurs	Docker, Kubernetes	Déploiement reproductible, montée en charge [4].
CI/CD	GitHub Actions	Intégré au dépôt, gratuit pour le projet.

Conclusion

Ce chapitre a défini les concepts du projet, comparé les solutions du marché et justifié nos choix technologiques. Aucune des solutions étudiées ne réunit le multi-cliniques, l'arabe et l'hébergement local. Le chapitre suivant spécifie les besoins de MediBook.

Chapitre 3

Analyse et spécification des besoins

Introduction

Ce chapitre identifie les acteurs de la plateforme, puis présente ses besoins fonctionnels et non fonctionnels. Il se termine par le diagramme de cas d'utilisation global et le backlog du produit. Le détail des exigences figure dans le cahier des charges.

3.1 Identification des acteurs

Tab. 3.1. Acteurs de la plateforme

Acteur	Rôle
Patient	Recherche un praticien, réserve, déplace ou annule un rendez-vous, consulte son historique.
Secrétaire	Gère les agendas des praticiens de sa clinique, confirme et déplace les rendez-vous.
Praticien	Définit ses horaires et ses absences, consulte son planning du jour.
Administrateur clinique	Gère les praticiens, les secrétaires, les motifs de consultation et l'abonnement de la clinique.
Super-administrateur	Membre de Nova Solutions : crée les cliniques, gère les offres et supervise la plateforme.
Service de notification	Système externe qui envoie les SMS et les e-mails de rappel.

3.2 Besoins fonctionnels

Tab. 3.2. Besoins fonctionnels par module

Module	Besoins principaux
Authentification	Inscription du patient, connexion, mot de passe oublié, second facteur pour le personnel.
Cliniques	Création d'une clinique, paramètres (adresse, horaires, logo), gestion du personnel et des rôles.

Module	Besoins principaux
Agendas	Horaires hebdomadaires des praticiens, absences, durée des créneaux par motif de consultation.
Réservation	Recherche par spécialité et par ville, choix d'un créneau, confirmation, déplacement, annulation.
Rappels	Rappel automatique 24 h avant le rendez-vous par SMS et par e-mail, confirmation de présence.
Tableau de bord	Taux d'occupation, rendez-vous non honorés, délai moyen d'attente, export CSV.
Abonnements	Offres mensuelles par nombre de praticiens, période d'essai, suspension en cas d'impayé.

3.3 Besoins non fonctionnels

Tab. 3.3. Besoins non fonctionnels

Besoin	Exigence
Sécurité	Données de santé chiffrées en transit et au repos ; contrôle d'accès par rôles.
Isolation	Une clinique ne peut jamais lire les données d'une autre clinique.
Performance	Recherche de créneaux en moins de 500 ms pour 95 % des requêtes.
Disponibilité	99,5 % par mois, hors maintenance planifiée.
Accessibilité	Écrans conformes WCAG 2.2 niveau AA [7].
Internationalisation	Français, anglais et arabe (écriture de droite à gauche).
Conformité	Respect de la loi tunisienne sur la protection des données personnelles.

3.4 Diagramme de cas d'utilisation global

La figure 3.1 présente les cas d'utilisation de la plateforme et leurs relations.

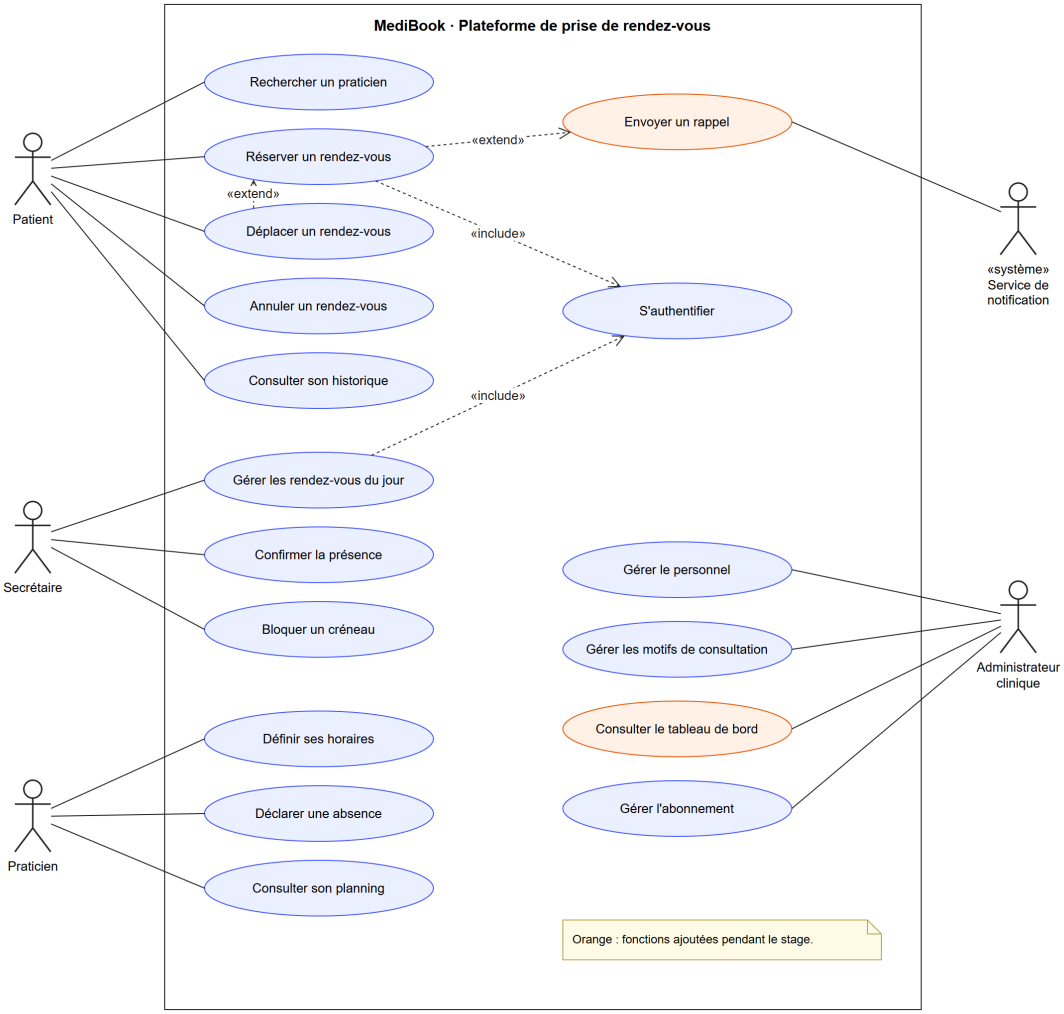


Fig. 3.1. Diagramme de cas d'utilisation global

3.5 Backlog du produit

Le tableau 3.4 présente un extrait du backlog, priorisé avec la méthode MoSCoW.

Tab. 3.4. Extrait du backlog du produit

ID	Histoire utilisateur	Priorité	Release
US1	En tant que patient, je veux créer un compte afin de réserver en ligne.	Must	R1
US2	En tant qu'administrateur clinique, je veux ajouter un praticien afin de publier son agenda.	Must	R1
US3	En tant que praticien, je veux définir mes horaires afin de générer mes créneaux.	Must	R1
US4	En tant que patient, je veux réserver un créneau libre afin d'éviter un appel.	Must	R2

ID	Histoire utilisateur	Priorité	Release
US5	En tant que patient, je veux recevoir un rappel afin de ne pas oublier mon rendez-vous.	Should	R2
US6	En tant que secrétaire, je veux déplacer un rendez-vous afin de gérer une urgence.	Should	R2
US7	En tant qu'administrateur clinique, je veux suivre le taux d'occupation afin d'ajuster les horaires.	Could	R3
US8	En tant que super-administrateur, je veux suspendre une clinique impayée.	Could	R3

Conclusion

Ce chapitre a identifié les acteurs, les besoins fonctionnels et non fonctionnels, puis présenté le diagramme de cas d'utilisation et le backlog. Le chapitre suivant décrit l'architecture et la conception qui répondent à ces besoins.

Chapitre 4

Architecture et conception

Introduction

Ce chapitre présente l'architecture globale de MediBook, puis sa conception détaillée : diagramme de classes, diagramme de séquence, diagramme d'activité et diagramme d'états.

4.1 Architecture globale

La figure 4.1 montre l'architecture en couches de la plateforme. Les applications React consomment une API REST Spring Boot, protégée par des jetons émis par Keycloak. Les rappels sont publiés dans RabbitMQ et envoyés par un service de notification indépendant.

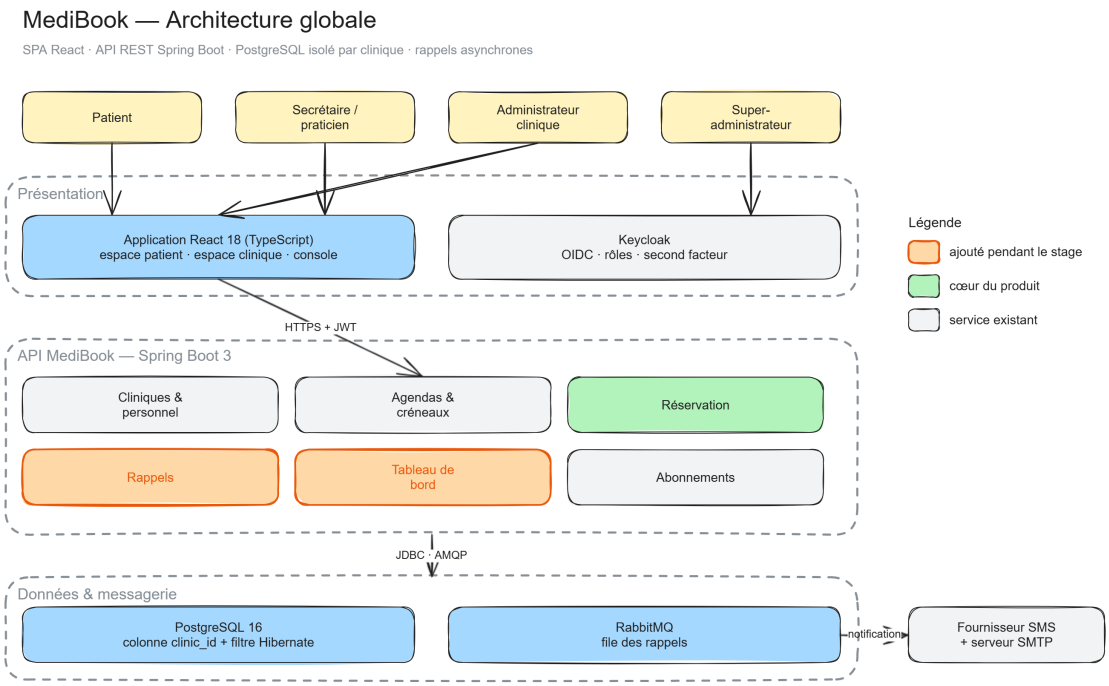


Fig. 4.1. Architecture globale de MediBook

4.1.1 Isolation des cliniques

Nous avons retenu la stratégie de la colonne `clinic_id` (tableau 2.1). Chaque requête porte l'identifiant de la clinique extrait du jeton ; un filtre Hibernate l'ajoute automatiquement à toutes les requêtes, et un test d'intégration vérifie qu'aucune donnée ne fuit d'une clinique à l'autre.

4.2 Conception détaillée

4.2.1 Diagramme de classes

La figure 4.2 présente les classes du domaine. Une clinique emploie des praticiens ; chaque praticien possède un agenda hebdomadaire, à partir duquel sont générés les créneaux réservables.

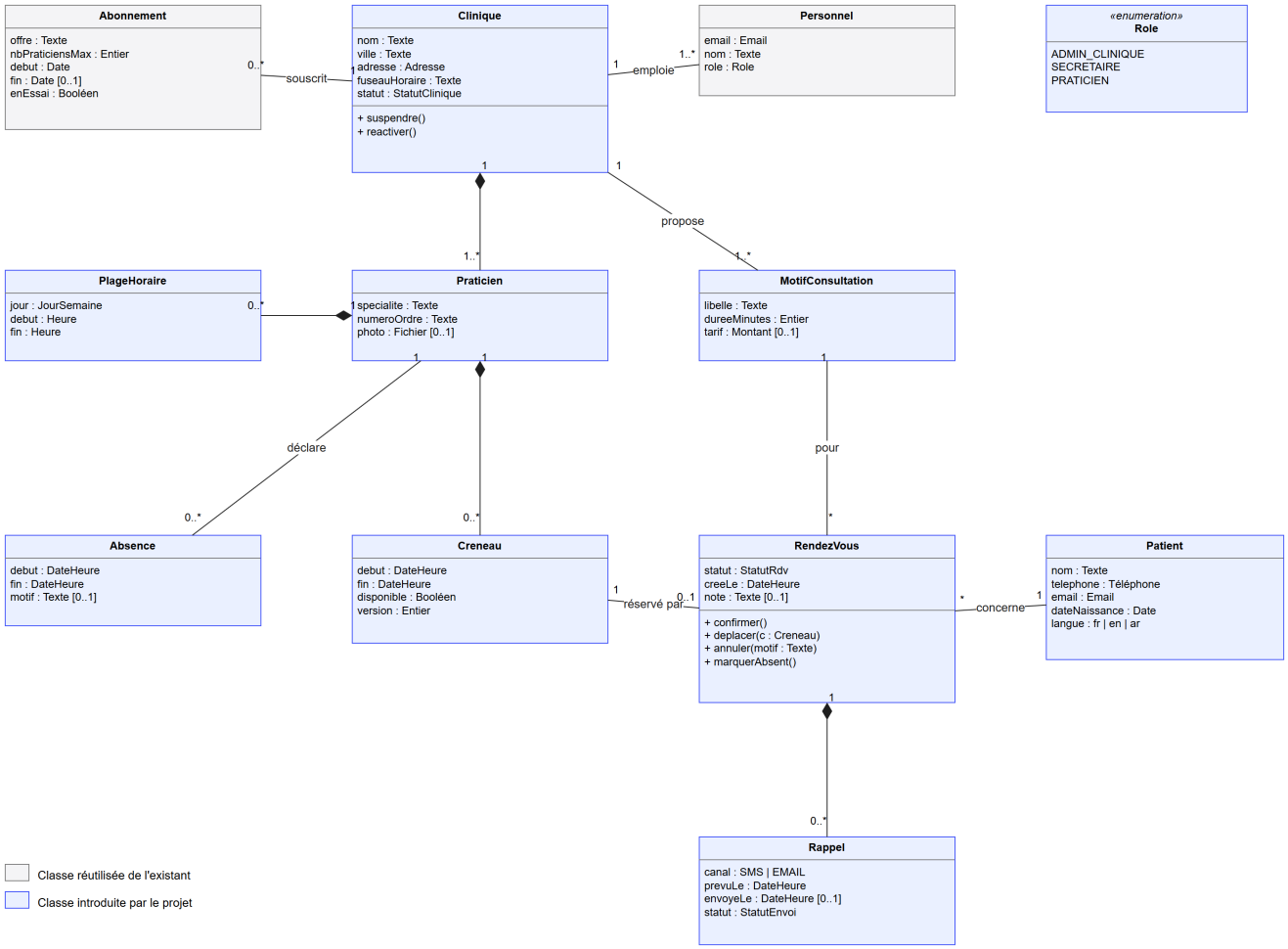


Fig. 4.2. Diagramme de classes d'analyse

4.2.2 Diagramme de séquence : prise de rendez-vous

La figure 4.3 décrit la réservation d'un créneau par un patient. Le verrou optimiste sur le créneau empêche deux patients de réserver le même horaire.

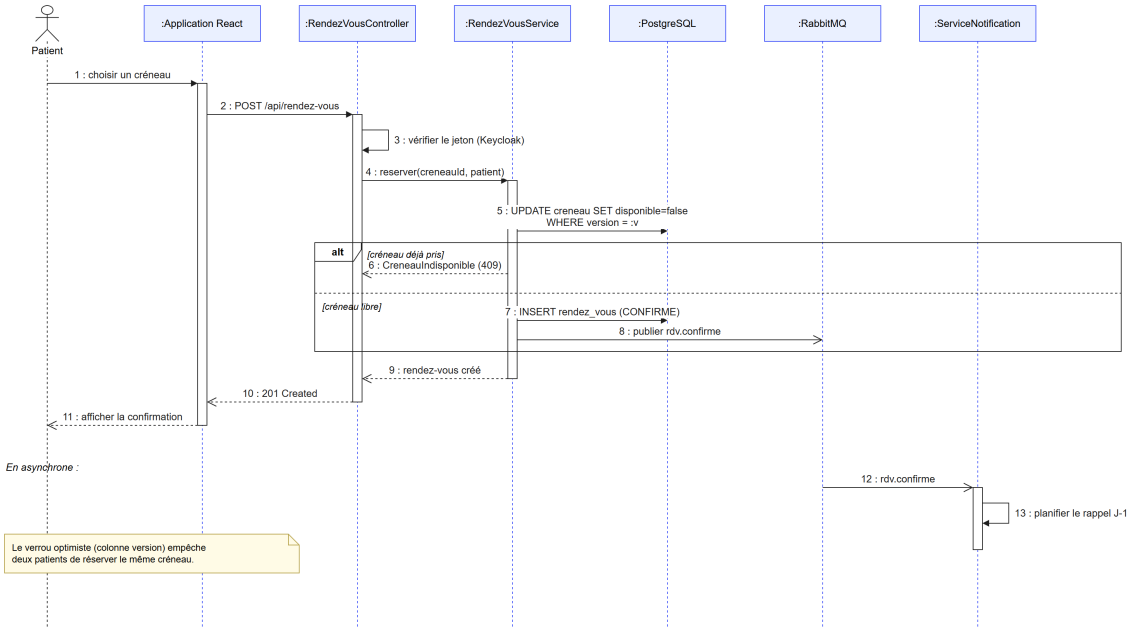


Fig. 4.3. Diagramme de séquence : prise de rendez-vous

4.2.3 Diagramme d'activité : prise de rendez-vous

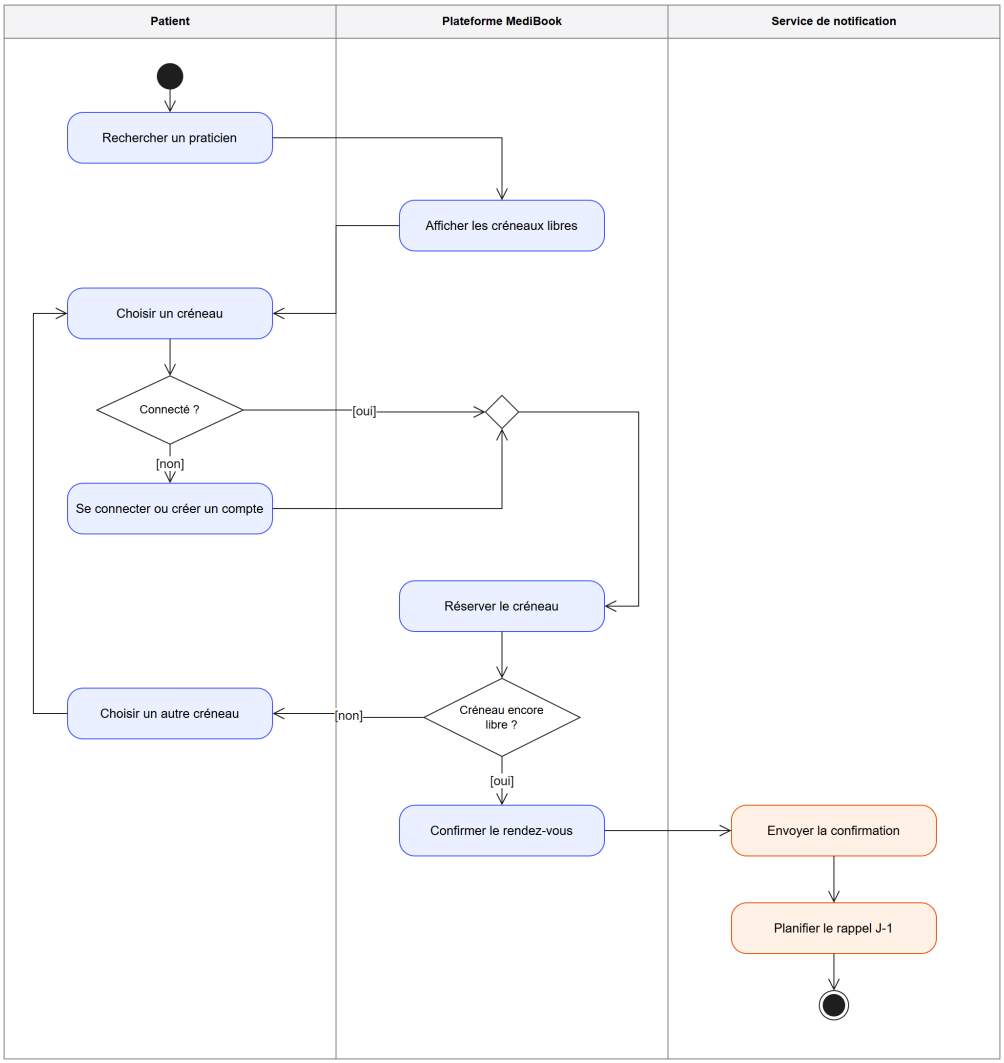


Fig. 4.4. Diagramme d'activité : prise de rendez-vous

4.2.4 Diagramme d'états : rendez-vous

La figure 4.5 présente le cycle de vie d'un rendez-vous, de sa création à sa clôture.

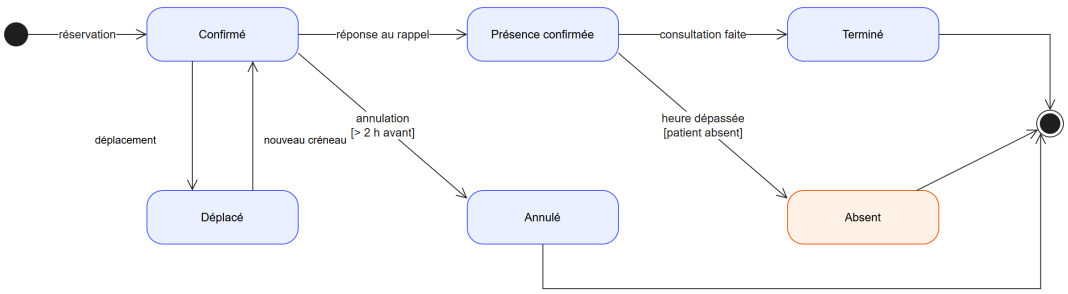


Fig. 4.5. Diagramme d'états d'un rendez-vous

Conclusion

Ce chapitre a présenté l'architecture de MediBook et sa conception détaillée. Le chapitre suivant décrit la réalisation de la solution, release par release.

Chapitre 5

Réalisation

Introduction

Ce chapitre présente l'environnement de travail, puis le travail réalisé dans chaque release, illustré par des captures d'écran de l'application.

5.1 Environnement de travail

Tab. 5.1. Environnement de développement

Outil	Usage
IntelliJ IDEA, VS Code	Développement back-end et front-end.
Git, GitHub	Gestion du code source, revues de code par *pull request*.
Jira	Suivi du backlog et des sprints.
Docker Desktop	Exécution locale de PostgreSQL, Keycloak et RabbitMQ.
Postman	Tests manuels de l'API.

5.2 Release 1 : socle et agendas

5.2.1 Backlog de la release

Cette release couvre les histoires US1 à US3 (tableau 3.4) : authentification, gestion des cliniques et des praticiens, et génération des créneaux. Les interfaces suivent le prototype validé en début de stage (figure C.2).

5.2.2 Interfaces réalisées

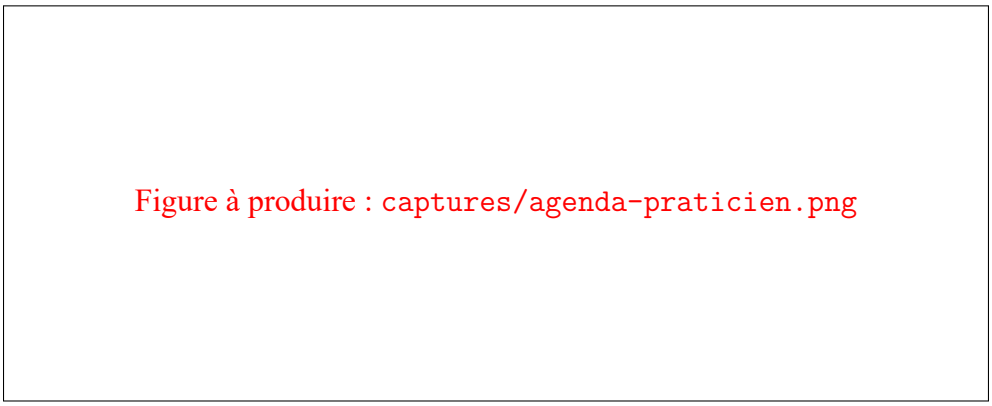


Fig. 5.1. Agenda hebdomadaire d'un praticien

[À compléter : Ajouter vos captures dans figures/captures/ : le cadre rouge disparaît dès que le fichier existe.]

5.3 Release 2 : réservation et rappels

5.3.1 Backlog de la release

Cette release couvre les histoires US4 à US6 : réservation par le patient, rappels automatiques et déplacement par le secrétariat.

5.3.2 Interfaces réalisées



Fig. 5.2. Réservation d'un créneau par le patient

5.4 Release 3 : pilotage et mise en production

5.4.1 Backlog de la release

Cette release couvre les histoires US7 et US8 : tableau de bord de la clinique, gestion des abonnements et déploiement sur Kubernetes.

5.4.2 Interfaces réalisées

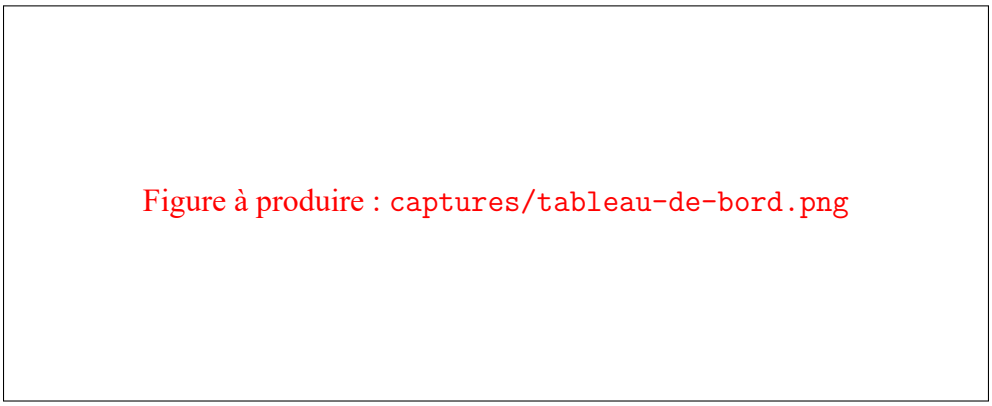


Fig. 5.3. Tableau de bord d'une clinique

Conclusion

Ce chapitre a présenté la réalisation de MediBook release par release. Le chapitre suivant décrit la stratégie de test et le déploiement de la solution.

Chapitre 6

Tests, qualité et déploiement

Introduction

Ce chapitre présente la stratégie de test, la chaîne d'intégration et de livraison continues, puis l'architecture de déploiement de la plateforme.

6.1 Stratégie de test

Tab. 6.1. Niveaux de test

Niveau	Outils	Périmètre
Unitaire	JUnit 5, Mockito, Vitest	Règles métier : génération des créneaux, chevauchements, annulation.
Intégration	Testcontainers	Dépôts JPA sur un vrai PostgreSQL, isolation des cliniques.
Bout en bout	Playwright	Parcours patient complet : recherche, réservation, annulation.
Charge	k6	200 utilisateurs simultanés sur la recherche de créneaux.

6.2 Résultats

Tab. 6.2. Résultats des tests (valeurs d'exemple)

Indicateur	Objectif	Mesure
Couverture du code back-end	≥ 80 %	84 %
Tests bout en bout réussis	100 %	100 % (36/36)
Temps de réponse (95e centile)	< 500 ms	310 ms
Vulnérabilités critiques (Trivy)	0	0

[À compléter : Remplacer par vos mesures réelles.]

6.3 Intégration et livraison continues

La figure 6.1 présente la chaîne CI/CD. Chaque *pull request* déclenche la compilation, les tests et l'analyse de qualité ; la fusion sur la branche principale construit les images Docker et les déploie sur le cluster.

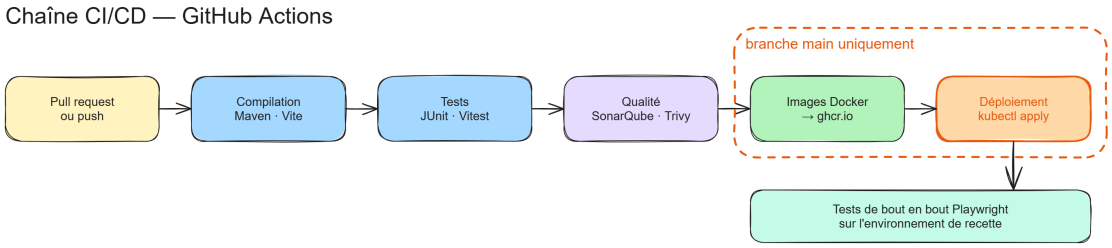


Fig. 6.1. Chaîne d'intégration et de livraison continues

6.4 Déploiement

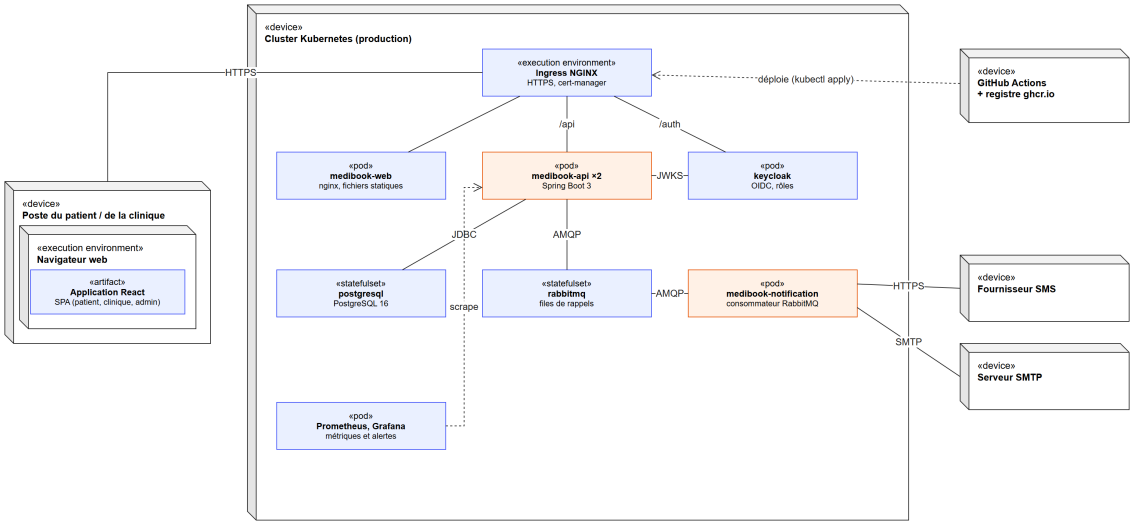


Fig. 6.2. Diagramme de déploiement

Conclusion

Ce chapitre a présenté la stratégie de test, les résultats obtenus, la chaîne CI/CD et l'architecture de déploiement de MediBook.

Conclusion générale et perspectives

Ce projet de fin d'études, réalisé au sein de Nova Solutions, avait pour objectif de concevoir et de réaliser MediBook, une plateforme SaaS de prise de rendez-vous en ligne pour les cliniques.

Après l'étude de l'existant et des solutions du marché, nous avons spécifié les besoins, conçu une architecture multi-cliniques fondée sur Spring Boot, React et PostgreSQL, puis réalisé la solution en trois releases. La plateforme permet aujourd'hui aux patients de réserver en ligne, aux secrétariats de gérer les agendas et aux cliniques de suivre leur activité. **[À compléter : Résumer les résultats chiffrés.]**

Ce stage nous a permis d'approfondir nos compétences en architecture logicielle, en sécurité des applications web et en DevOps, et de travailler au sein d'une équipe agile.

Plusieurs perspectives s'ouvrent à ce travail : la téléconsultation par vidéo, une application mobile pour les patients, et l'intégration avec les dossiers patients informatisés des cliniques.

Bibliographie et netographie

[1] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

[2] Meta Open Source. React documentation. `https://react.dev/`, 2026. Consulté le 12 février 2027.

[3] Ken Schwaber and Jeff Sutherland. The scrum guide. `https://scrumguides.org/scrum-guide.html`, 2020. Consulté le 10 février 2027.

[4] The Kubernetes Authors. Kubernetes documentation. `https://kubernetes.io/docs/`, 2026. Consulté le 14 février 2027.

[5] The PostgreSQL Global Development Group. Postgresql 16 documentation. `https://www.postgresql.org/docs/16/`, 2026. Consulté le 12 février 2027.

[6] VMware Tanzu. Spring boot reference documentation. `https://docs.spring.io/spring-boot/`, 2026. Consulté le 12 février 2027.

[7] W3C. Web content accessibility guidelines (wcag) 2.2. `https://www.w3.org/TR/WCAG22/`, 2023. Consulté le 14 février 2027.

Chapitre A

Planning détaillé et rendez-vous pédagogiques

Tab. A.1. Planning détaillé du stage (26 semaines)

Semaine	Travail et jalons
1–2	Sprint 0 : étude de l'existant, état de l'art, cahier des charges
3	**Plan de travail validé** par les deux encadrants
3–10	Release 1 : authentification, cliniques, praticiens, agendas
6	**Bilan périodique de début de stage**
9	**1re restitution** à ESPRIT
11–18	Release 2 : espace patient, réservation, rappels
13–14	**Visite de mi-parcours** de l'encadrant académique
15	**Bilan périodique de milieu de stage**
18	**Dépôt de la version 0 du rapport**
19–24	Release 3 : tableau de bord, abonnements, déploiement Kubernetes
21	**2e restitution** et évaluation par l'expert
23	**Bilan périodique de fin de stage**
25–26	Recette ; **dépôt de la version finale du rapport**
Chaque semaine	Mise à jour du **journal de bord**

Chapitre B

Extraits de configuration

B.1 Filtre d'isolation des cliniques

Le filtre suivant ajoute automatiquement la clinique courante à toutes les requêtes JPA.

```
@FilterDef(name = "clinicFilter",
           parameters = @ParamDef(name = "clinicId", type = UUID.class))
@Filter(name = "clinicFilter", condition = "clinic_id = :clinicId")
@MappedSuperclass
public abstract class ClinicScopedEntity {
    @Column(name = "clinic_id", nullable = false, updatable = false)
    private UUID clinicId;
}
```

B.2 Déploiement Kubernetes

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: medibook-api
spec:
  replicas: 2
  template:
    spec:
      containers:
        - name: api
          image: ghcr.io/exemple/medibook-api:1.0.0
          ports: [{ containerPort: 8080 }]
          readinessProbe:
            httpGet: { path: /actuator/health/readiness, port: 8080 }
```

Chapitre C

Design Thinking : brainstorming et prototype

C.1 Séance de brainstorming

La figure C.1 présente le tableau de la séance de brainstorming du Sprint 0, organisée avec l'équipe de Nova Solutions. Les idées sont regroupées en dix thèmes autour de l'idée centrale du projet ; elles ont alimenté l'étude de l'existant (chapitre 1) et le backlog du produit (chapitre 3).

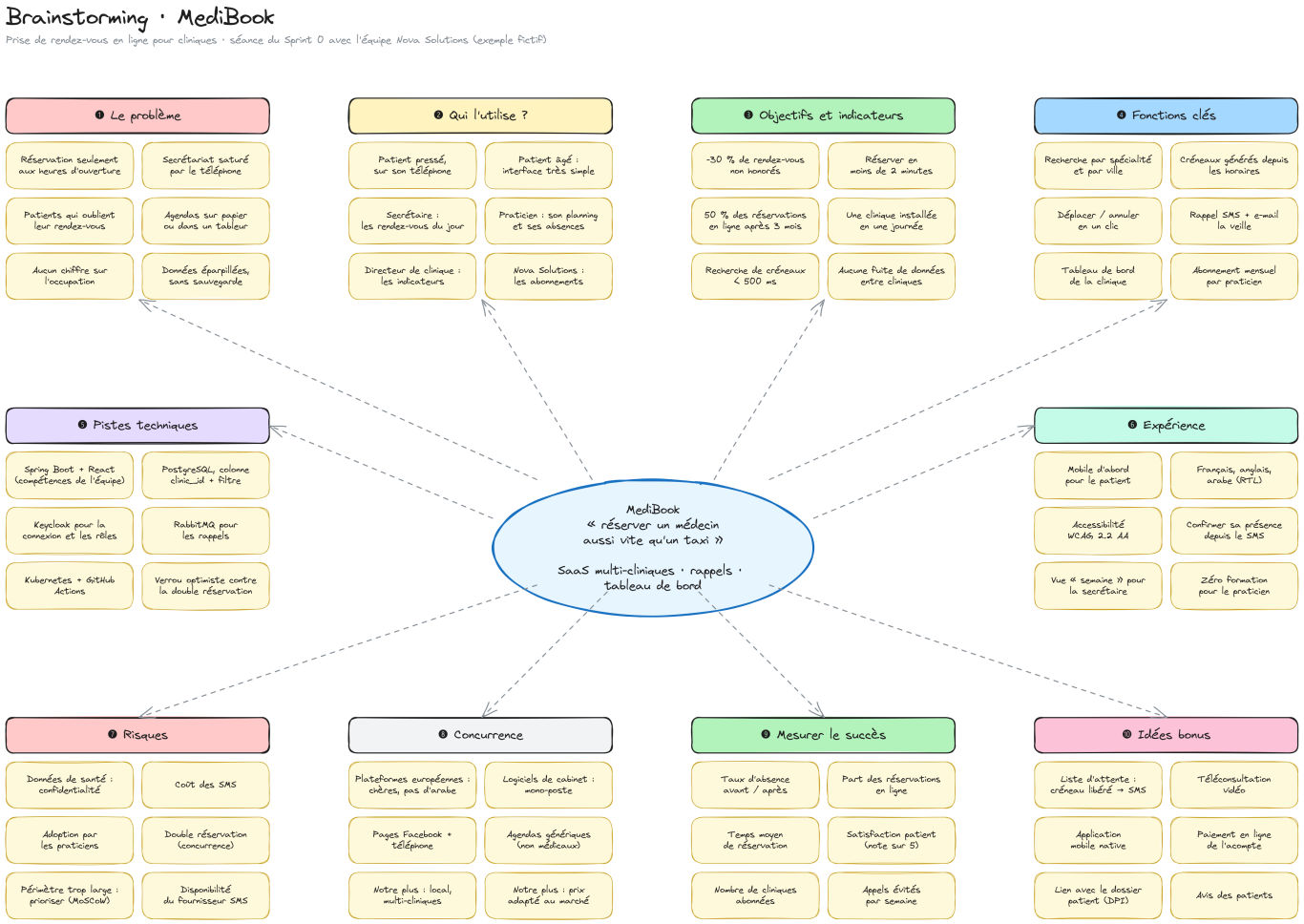


Fig. C.1. Tableau de brainstorming de MediBook

C.2 Prototype basse fidélité

La figure C.2 présente les maquettes basse fidélité validées avec l'encadrant entreprise avant le développement : le parcours de réservation du patient sur mobile (A1 à A4), l'espace clinique (B1, B2), le tableau de bord et la console d'administration (C1, C2). Les écrans réalisés sont présentés au chapitre 5.

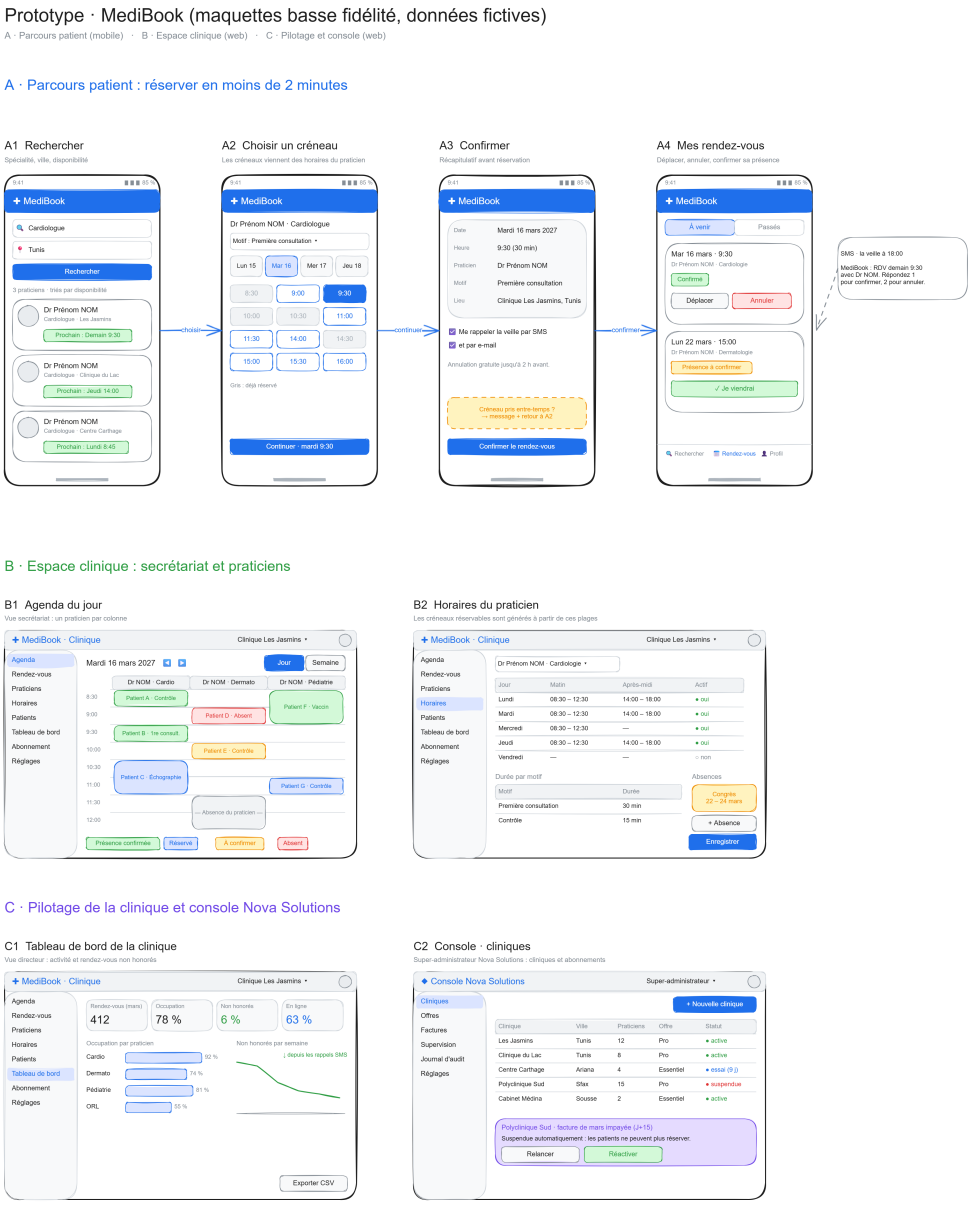


Fig. C.2. Prototype basse fidélité de MediBook

Résumé

Ce projet de fin d'études, réalisé au sein de Nova Solutions, porte sur la conception et la réalisation de MediBook, une plateforme SaaS multi-cliniques de prise de rendez-vous en ligne et de suivi des patients. La solution permet aux patients de réserver un créneau sans appel téléphonique, aux secrétariats de gérer les agendas des praticiens et aux cliniques de réduire les rendez-vous non honorés grâce à des rappels automatiques par SMS et par e-mail. Elle repose sur une architecture Spring Boot et React, une base PostgreSQL isolée par clinique, une authentification centralisée avec Keycloak et un déploiement conteneurisé sur Kubernetes, livré par une chaîne d'intégration continue. **[À compléter : Compléter avec les résultats chiffrés.]**

Mots-clés : SaaS, e-santé, prise de rendez-vous, multi-location, Spring Boot, React, PostgreSQL, Keycloak, Kubernetes, Scrum.

Abstract

This end-of-studies project, carried out at Nova Solutions, covers the design and implementation of MediBook, a multi-clinic SaaS platform for online appointment booking and patient follow-up. Patients book a slot without a phone call, front desks manage the practitioners' calendars, and clinics reduce missed appointments thanks to automatic SMS and e-mail reminders. The solution relies on a Spring Boot and React architecture, a PostgreSQL database isolated per clinic, centralised authentication with Keycloak and a containerised deployment on Kubernetes, delivered by a continuous integration pipeline. **[À compléter : Add the measured results.]**

Keywords : SaaS, e-health, appointment booking, multi-tenancy, Spring Boot, React, PostgreSQL, Keycloak, Kubernetes, Scrum.